Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

**Лабораторная работа №1**

По дисциплине: «АПО ЭВМиС»

Тема: «Изучение аппаратно-программной архитектуры процессоров семейства Intel – разработка ассемблерной вставки»

Выполнил:
Студент 4-го курса
Группы АС-66
Куган Н. Л.
Проверила:
Кулик А. Д.

Брест 2026

**Цель работы:** изучение программной модели микропроцессора, регистров, организации памяти; практическая работа с ассемблерными вставками.

## Задание

1. Изучить теоретические сведения: программная модель микропроцессора, регистры и их назначение, организация памяти, основные конструкции и команды ассемблера.

2. Написать ассемблерную вставку, реализующую обработку строки согласно варианту, и оформить её в виде отдельной функции.

3. Реализовать ту же обработку в виде функции на C++ и сравнить быстродействие обеих реализаций.

## Вариант 9

Перевернуть две половины строки. Каждая половина (длиной n / 2) разворачивается на месте независимо от другой, у строки нечётной длины средний символ остаётся на месте. Пример: «abcdef» → «cbafed», «abcde» → «baced».

## Реализация

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <string>
#include <chrono>
#include <cstring>
#include <algorithm>

// 1. Реализация на C++
void reverseRange(char* s, size_t count) {
    for (size_t i = 0; i < count / 2; ++i)
        std::swap(s[i], s[count - 1 - i]);
}

void reverseHalvesCpp(char* s) {
    size_t n = strlen(s), half = n / 2;
    reverseRange(s, half);              // первая половина
    reverseRange(s + n - half, half);   // вторая половина
}

// 2. Реализация на ассемблере (MSVC, x86)
void reverseHalvesAsm(char* s) {
    __asm {
        push ebx
```

```
        push esi
        push edi
        mov esi, s              ; esi = начало строки
        mov edi, esi
    find_end:
        cmp byte ptr [edi], 0
        je end_found
        inc edi
        jmp find_end
    end_found:                  ; edi = адрес завершающего нуля
        mov ecx, edi
        sub ecx, esi
        shr ecx, 1              ; ecx = длина половины
        jz done
        mov eax, esi            ; eax = левый указатель
        lea edx, [esi + ecx - 1] ; edx = правый указатель
    loop1:                      ; первая половина
        cmp eax, edx
        jae part2
        mov bl, [eax]
        mov bh, [edx]
        mov [eax], bh
        mov [edx], bl
        inc eax
        dec edx
        jmp loop1
    part2:
        mov eax, edi
        sub eax, ecx
        lea edx, [edi - 1]
    loop2:                      ; вторая половина
        cmp eax, edx
        jae done
        mov bl, [eax]
        mov bh, [edx]
        mov [eax], bh
        mov [edx], bl
        inc eax
        dec edx
        jmp loop2
    done:
        pop edi
        pop esi
        pop ebx
    }
}


// Среднее время (мс) пяти запусков на копии строки
double bench(void (*f)(char*), const std::string& src) {
    static volatile char sink;
    double total = 0;
    for (int r = 0; r < 5; ++r) {
        std::string w = src;
        auto t0 = std::chrono::high_resolution_clock::now();
        f(&w[0]);
        auto t1 = std::chrono::high_resolution_clock::now();
        total += std::chrono::duration<double, std::milli>(t1 - t0).count();
        sink = w[1];
    }
    return total / 5;
}


int main() {
    const char* tests[] = { "", "a", "abcd", "abcde", "abcdef", "Hello, World!" };
    for (const char* t : tests) {
        char a[64], b[64];
        strcpy(a, t);
        strcpy(b, t);
```

```
        reverseHalvesCpp(a);
        reverseHalvesAsm(b);
        std::cout << "\"" << t << "\" -> C++: \"" << a << "\", Asm: \"" << b << "\" "
                  << (strcmp(a, b) == 0 ? "OK" : "ERROR") << std::endl;
    }

    // Сравнение быстродействия на большой строке
    const size_t N = 16000000;
    std::string big(N, 'a');
    for (size_t i = 0; i < N; ++i) big[i] = 'a' + i % 26;
    std::cout << "Time C++: " << bench(reverseHalvesCpp, big) << " ms" << std::endl;
    std::cout << "Time Asm: " << bench(reverseHalvesAsm, big) << " ms" << std::endl;
    return 0;
}
```

## Результат работы программы

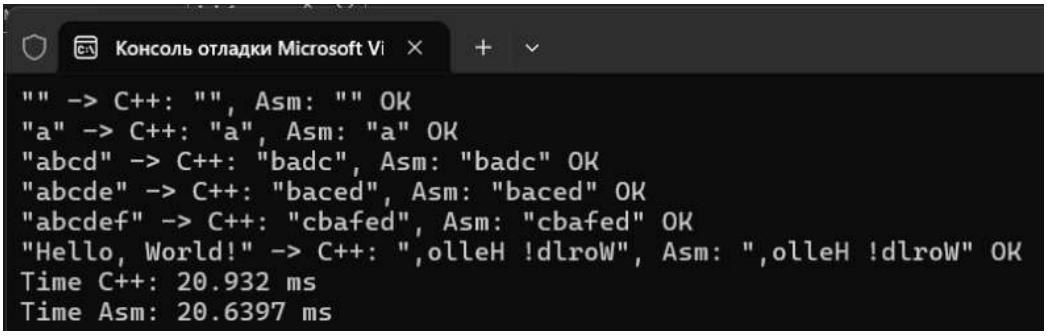



Рисунок 1 – Результат работы программы

**Вывод**: В ходе работы изучены программная модель микропроцессора Intel x86, регистры общего назначения, адресация памяти и порядок написания ассемблерной вставки __asm в программе на C++. Для варианта 9 написаны две функции, разворачивающие две половины строки: на ассемблере и на C++. Обе функции дали одинаковые результаты на всех тестовых строках и на строке из 16 млн символов. Сравнение быстродействия (сборка Release, платформа x86) показало практически одинаковое время: 20,93 мс у реализации на C++ и 20,64 мс у ассемблерной вставки, разница около 1,4 % находится в пределах погрешности измерения. Это объясняется тем, что оптимизирующий компилятор строит для C++-кода цикл не хуже написанного вручную, а на строке такого размера скорость ограничена обменом с памятью. Ассемблерная вставка даёт полный контроль над регистрами и памятью, но проигрывает C++ в читаемости и переносимости, поэтому оправдана только для критичных по скорости участков.